

TRATAMENTO DE ERROS E EXCEÇÕES

Prof. André Backes

Tratamento de erros e exceções

- Nenhum sistema é perfeito
- Todo sistema está sujeito a ações que causam anomalias
 - Divisão por zero
 - Raiz quadrada de um número negativo
 - Abrir um arquivo que não existe
 - ...

Tratamento de erros e exceções

- Em algum momento o sistema terá que tratar uma exceção, e é melhor ele estar preparado para isso
 - Ou o usuário terá seu sistema encerrado de forma inesperada
 - Ou receberá uma mensagem incompreensível
- Assim, um bom sistema precisa tratar essas exceções

Tratamento de erros e exceções

- O tratamento de exceções permite capturar erros ocorridos durante a execução de um programa
 - Para tanto, um programa pode “lançar” e “capturar” exceções

Tratamento de erros e exceções

- Na linguagem C++ o tratamento de erros e exceções é feito usando apenas 3 palavras chaves
- São elas:
 - try
 - catch
 - throw

Comando throw

- Esse comando gera uma exceção personalizada quando um problema é detectado no código
- Forma geral:

throw expressão;

- onde
 - expressão é o valor associado a exceção gerada
 - Pode ser de qualquer tipo: int, double, char, string etc.

Comando throw

- Basicamente, o comando **throw**
 - Interrompe o fluxo do código que está sendo executado
 - Lança uma exceção para ser tratada por um código definido pelo programador

Comando throw

- Esse comando pode ser usado diretamente dentro de um bloco **try**
 - Este é o bloco que irá verificar a existência de uma exceção

```
try {  
    if(y < 0 || y > 100)  
        throw 42;  
}  
catch(...){  
    cout << "Valor proibido!";  
}
```


Comando throw

- Esse comando pode ser usado em trechos delimitados do código, como em funções, permitindo um uso mais modularizado

```
double raiz(double x){  
    if (x < 0.0)  
        throw "Erro: valor negativo";  
  
    return sqrt(x);  
}  
  
int main(){  
    double x, y;  
    cout << "Digite um numero: ";  
    cin >> x;  
    try{  
        y = raiz(x);  
        cout << "Raiz = " << y << endl;  
    }  
    catch(const char *erro){  
        cout << erro;  
    }  
  
    return 0;  
}
```

Bloco try-catch

- Para tratar uma exceção é necessário definir um bloco **try-catch**
- Sua forma geral é:

```
try {  
    /// bloco de comando para ser testado  
}  
catch(...) {  
    /// bloco de comando para ser executado  
    /// no caso de uma exceção ser lançada  
}
```

Bloco try-catch

- Bloco **try**

- Define um trecho do programa onde um erro pode acontecer
- É aqui que o comando **throw** irá lançar a sua exceção

- Bloco **catch**

- Irá capturar essa exceção
- Contém o código que será executado em caso de erro

```
try{  
    /// bloco de comandos a ser testado  
  
    throw 42; ///gera uma exceção personalizada  
}  
catch(...){  
    /// Código a ser executado em caso de erro  
}
```

Bloco try-catch

- Um bloco **catch** pode estar associado a qualquer tipo de erro ou a um erro específico
 - Quando o tipo de erro não importa, colocamos três pontos (...) dentro dos parênteses do comando **catch**
 - Para um código de erro personalizado, especificamos a variável associada a esse erro dentro dos parênteses do comando **catch**

Bloco try-catch

Tratando um erro genérico

```
int idade;  
cout << "Digite a idade: ";  
cin >> idade;  
  
try{  
    if(idade < 18)  
        throw 0;  
}  
catch(...){  
    cout << "Idade invalida!";  
}
```

Tratando um erro específico

```
int idade;  
cout << "Digite a idade: ";  
cin >> idade;  
  
try{  
    if(idade < 18)  
        throw 123;  
}  
catch(int erro){  
    cout << "Idade invalida!\n";  
    cout << "Erro: " << erro;  
}
```

Bloco try-catch

- Não é recomendado o fazer tratamento de exceções dentro dos destrutores de classes
 - Isso pode atrapalhar a liberação da pilha de processos do programa

Manipulando múltiplos erros

- Um único trecho de código pode estar sujeito a vários tipos de problemas
 - Depende da complexidade do nosso programa
- Um bloco **try** pode possuir vários blocos **catch**
 - Um bloco **try** pode lançar vários tipos diferentes de exceções
 - Cada bloco **catch** trata um tipo de exceção específica

```
try{
    if(idade < 18)
        throw 123;
    else
        throw 5.1;
}
catch(int A){
    cout << "Erro inteiro: " << A;
}
catch(double B){
    cout << "Erro double: " << B;
}
catch(...){
    cout << "Erro default";
}
```

Manipulando múltiplos erros

- No exemplo, duas exceções específicas e uma genérica são tratadas
 - O tipo da exceção é comparado, na ordem, com os definidos pelo comando **catch**
 - Executa a sequência de comandos do bloco **catch** com tipo igual
 - Caso nenhum tipo seja igual, executa a sequência de comandos do **catch** genérico

```
try{
    if(idade < 18)
        throw 123;
    else
        throw 5.1;
}
catch(int A){
    cout << "Erro inteiro: " << A;
}
catch(double B){
    cout << "Erro double: " << B;
}
catch(...){
    cout << "Erro default";
}
```


Manipulando múltiplos erros

- O nome da variável pode ser omitido se o parâmetro não for usado no bloco **catch**
 - A seleção é feita pelo tipo
 - O nome da variável somente será necessário se ela for utilizada dentro do bloco **catch**

```
try{  
    if(idade < 18)  
        throw 123;  
    else  
        throw 5.1;  
}  
catch(int){  
    cout << "Erro inteiro";  
}  
catch(double){  
    cout << "Erro double";  
}  
catch(...){  
    cout << "Erro default";  
}
```

Aninhamento de exceções

- Também é possível aninhar blocos **try-catch**
 - O comando **throw** pode ser usado para passar adiante um erro
 - Neste caso, deve ser usado sem argumentos

```
try{
    cout << "Bloco try externo\n";
    try{
        cout << "Bloco try interno\n";
        throw 42;
    }catch(int erro){
        cout << "Bloco catch interno\n";
        cout << "Erro: " << erro << endl;
        throw;
    }
}catch(int codigo){
    cout << "Bloco catch externo\n";
    cout << "Erro: " << codigo;
}
```

Exceções padronizadas

- Biblioteca **exception**, dentro do namespace **std**
 - Biblioteca criada especificamente para tratar exceções
- Dentro dela temos a classe **exception**
 - Permite declarar objetos para ser lançados como exceção
 - Também possui um método virtual **what()**
 - Retorna uma sequência de caracteres (do tipo char*) contendo a descrição da exceção

Exceções padronizadas

- Alguns tipos de exceção presentes na classe **exception**

Exceção	Descrição
bad_alloc	lançada quando o operador new falha
bad_cast	lançada quando uma operação de <i>casting</i> falha
bad_exception	lançada por alguns especificadores de exceção dinâmico
bad_typeid	lançada quando um ponteiro para um objeto polimórfico possui valor nulo
bad_function_call	lançada por objetos de funções quando estes falham
bad_weak_ptr	lançada quando o construtor de um ponteiro compartilhado falha

Exceções padronizadas

- Exemplo de exceção do tipo **bad_alloc**
 - Lançada quando o operador **new** falha

```
#include <iostream>
#include <exception>
using namespace std;

int main() {
    try{
        int *V = new int[10000000000];
    }
    catch(exception& e) {
        cout << "Erro: " << e.what();
    }
    return 0;
}
```

Material Complementar

- Aula 21 - Tratamento de erros e exceções
 - <https://youtu.be/EpgIMpjZsFo>